MEMORIA TÉCNICA · ASIR

ORM con SQLAlchemy y MariaDB

Persistencia de objetos Python en un SGBD relacional y análisis del SQL generado

Ubuntu Server · MariaDB · Python 3 · SQLAlchemy 2 · PyMySQL

Adrián Sayas Descalzo

Contenido

1	Resumen y objetivo
2	Entorno y arquitectura
3	Preparación de MariaDB
4	Base de datos, usuario y registro
5	Entorno Python
6	Modelo ORM y creación de tablas
7	Persistencia y consulta de objetos
8	SQL generado y mínimo privilegio
9	Resultado y competencias
10	Conclusión

1. Resumen y objetivo

En este laboratorio trabajé con el desfase objeto-relacional utilizando SQLAlchemy como ORM sobre MariaDB. Preparé el servidor, creé una base de datos y un usuario de aplicación con permisos limitados, configuré un entorno Python y comprobé cómo el ORM transforma clases y objetos en tablas, filas y sentencias SQL.

El interés desde administración de SGBD está en observar qué tablas y sentencias genera el ORM y con qué permisos se conecta la aplicación.

2. Entorno y arquitectura

Elemento	Configuración / función
Sistema	Ubuntu Server 24.04
SGBD	MariaDB 10.11
Lenguaje	Python 3
ORM	SQLAlchemy 2
Conector	PyMySQL
Base de datos	instituto
Usuario de aplicación	app_instituto

El laboratorio separa las responsabilidades del servidor de base de datos y de la aplicación Python. La aplicación se conecta mediante un usuario técnico específico y el servidor registra las consultas para poder observar el SQL generado por el ORM.

3. Preparación de MariaDB

3.1 Comprobación del sistema

Comprobé la versión de Ubuntu y el usuario de trabajo antes de instalar las herramientas necesarias.

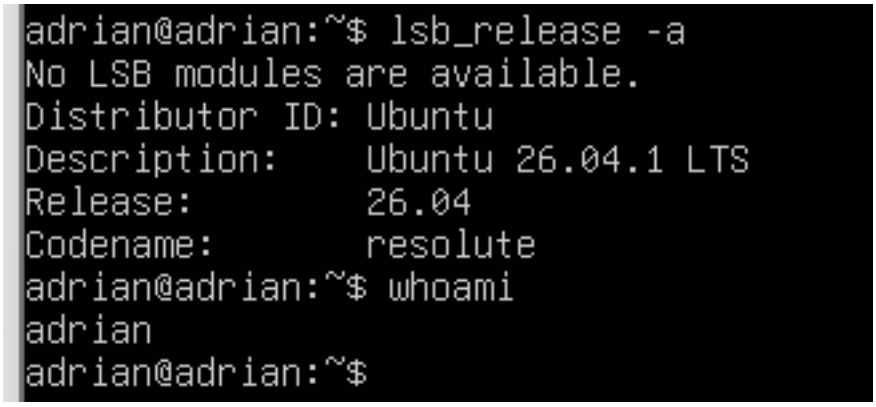


Figura 1. Versión de Ubuntu y usuario de trabajo.

3.2 Actualización de paquetes

Actualicé el índice de paquetes antes de instalar el software necesario para la práctica.

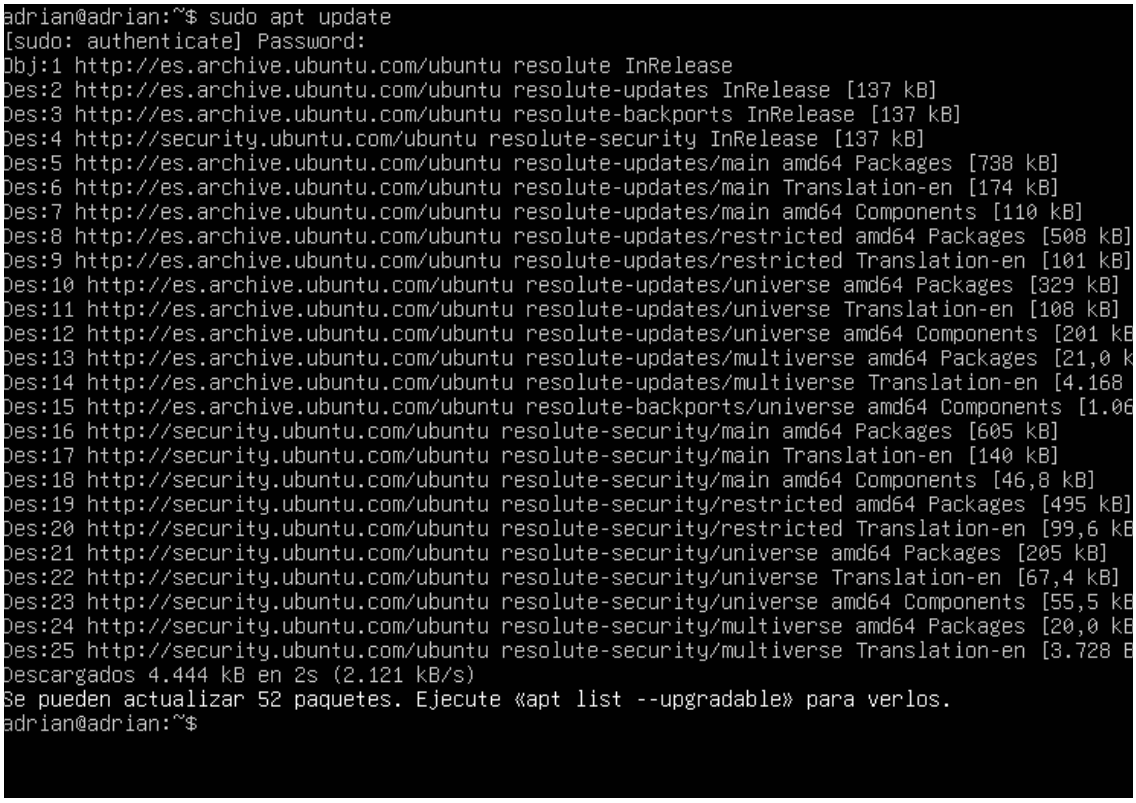


Figura 2. Actualización del índice de paquetes.

3.3 Instalación de MariaDB y Python

Instalé MariaDB junto con python3-venv, necesario para aislar posteriormente las dependencias Python.

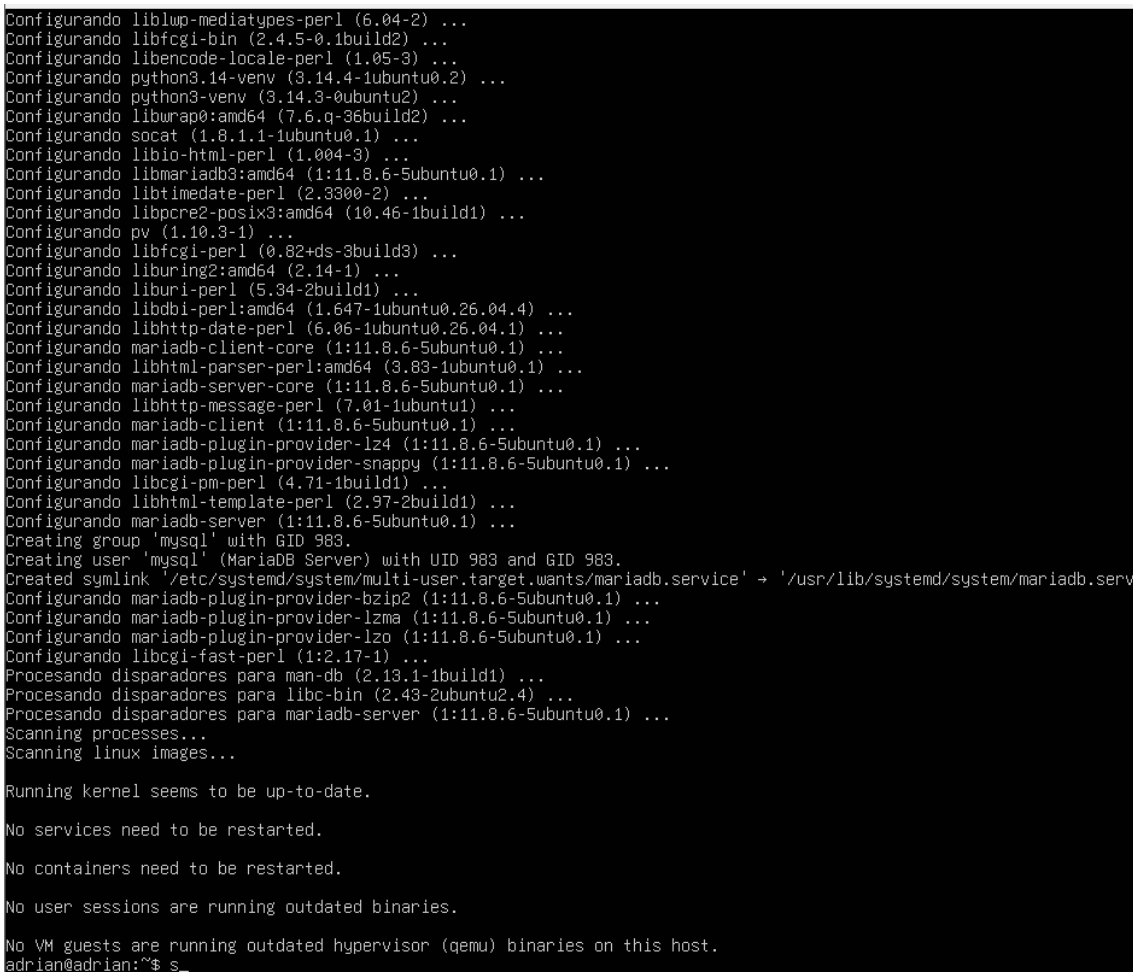


Figura 3. Instalación de MariaDB y python3-venv.

3.4 Estado del servidor

Comprobé la versión del cliente y que el servidor MariaDB estuviera operativo antes de crear la base de datos de la aplicación.



Figura 4. Versión del cliente y estado del servidor MariaDB.

4. Base de datos, usuario de aplicación y registro

4.1 Creación de la base de datos

Creé la base de datos instituto con codificación utf8mb4 y comprobé que aparecía correctamente en el servidor.

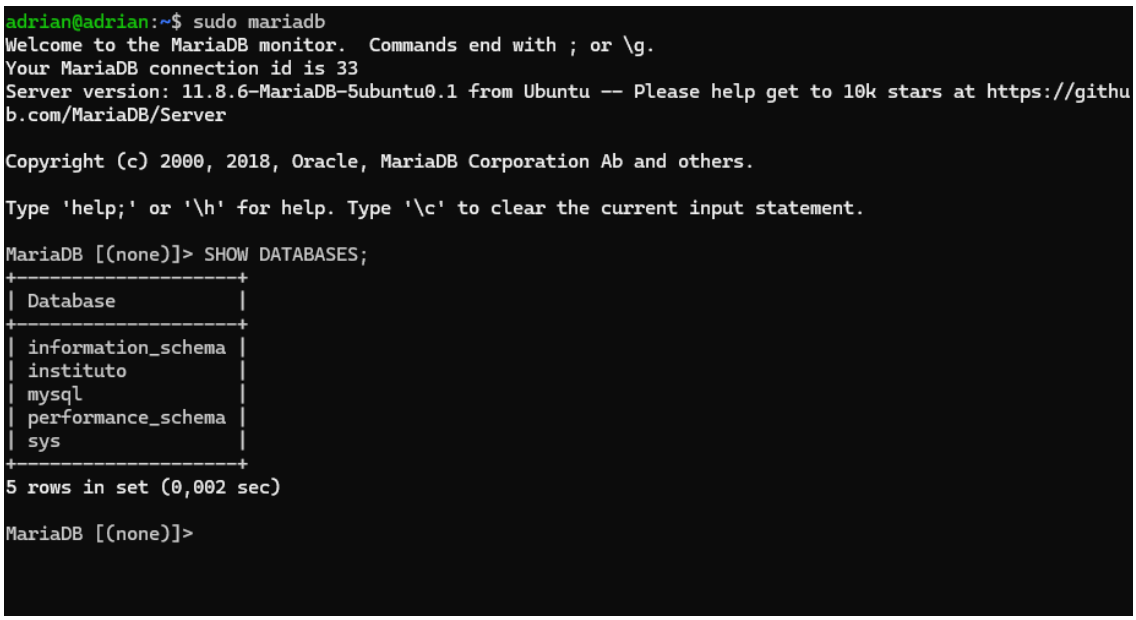


Figura 5. Creación y comprobación de la base de datos instituto.

4.2 Usuario técnico y privilegios

Creé el usuario app_instituto para la aplicación y revisé sus privilegios, limitándolos a las operaciones necesarias sobre la base de datos instituto.

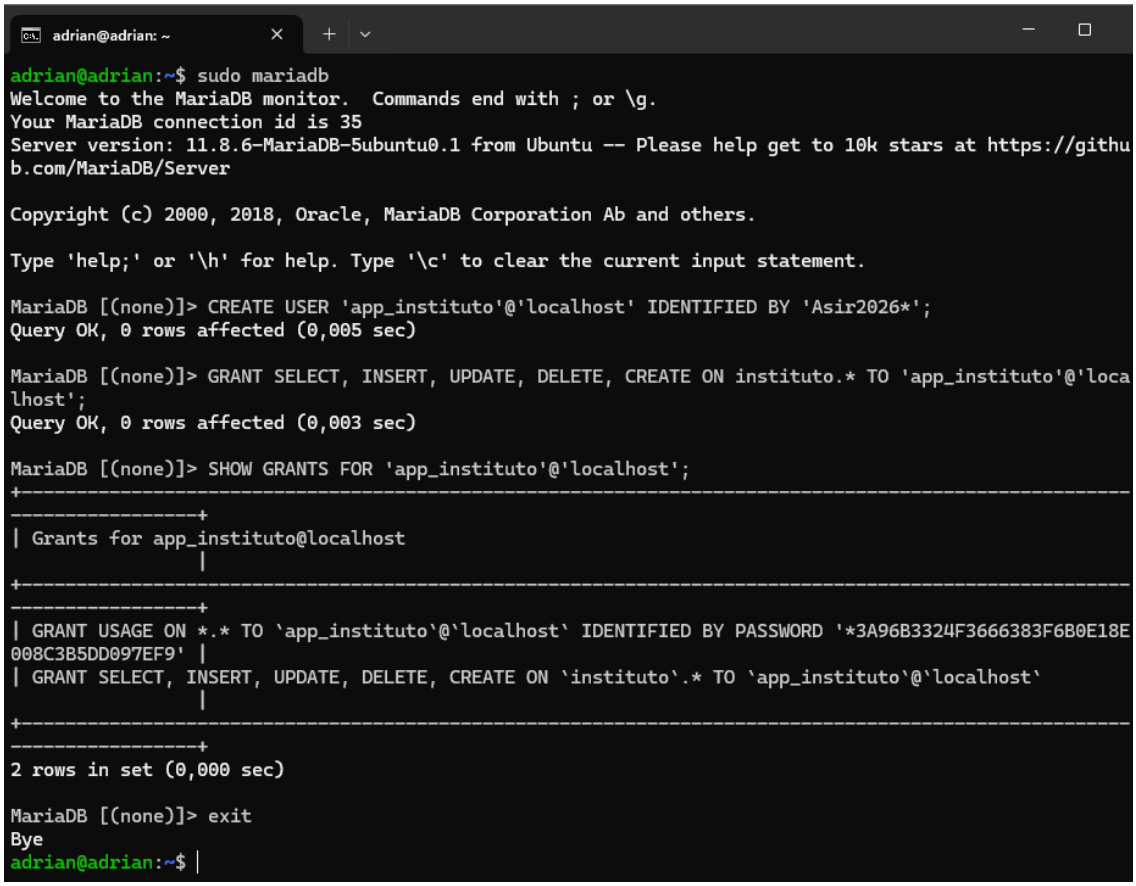


Figura 6. Creación del usuario app_instituto y revisión de sus privilegios.

4.3 Registro general de consultas

Activé el general log de MariaDB para registrar las sentencias que el ORM enviaría al servidor y poder analizarlas posteriormente.

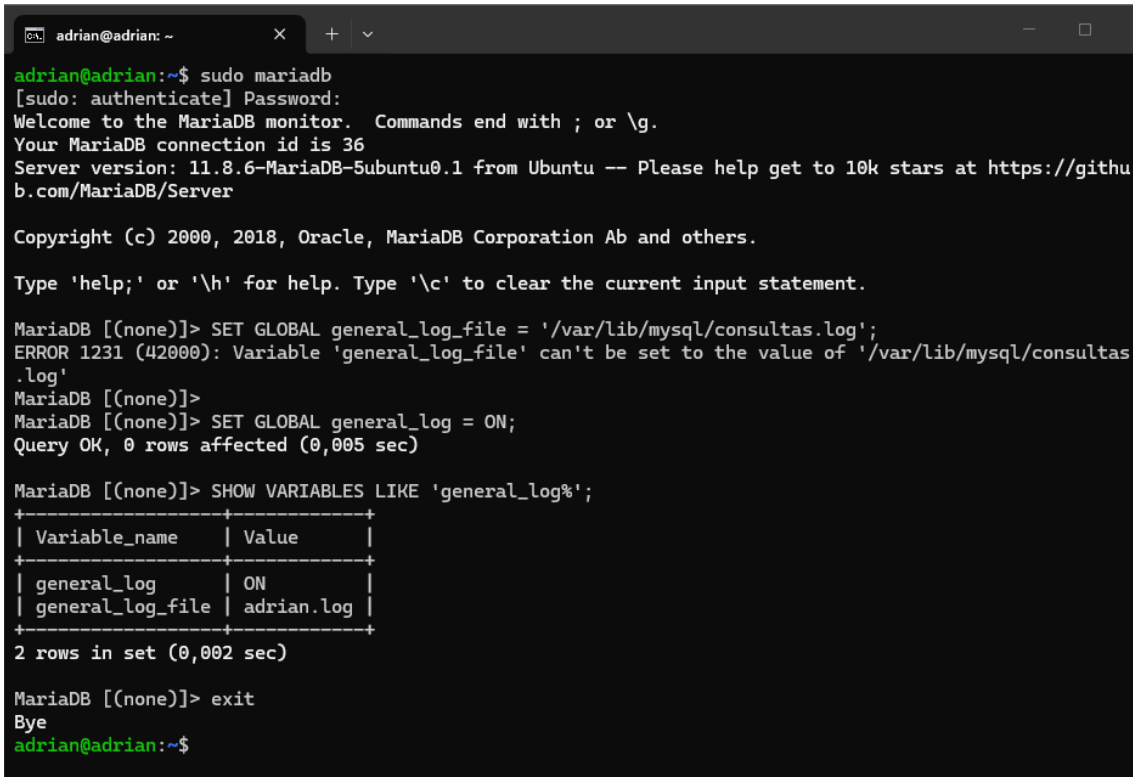


Figura 7. Activación y comprobación del registro general de consultas.

5. Entorno Python

Preparé un entorno virtual de Python para mantener aisladas las dependencias de la práctica. Dentro de ese entorno instalé SQLAlchemy y PyMySQL y verifiqué los paquetes disponibles.

5.1 Creación del entorno virtual

Creé el directorio de trabajo y activé el entorno virtual de Python utilizado por los scripts del laboratorio.

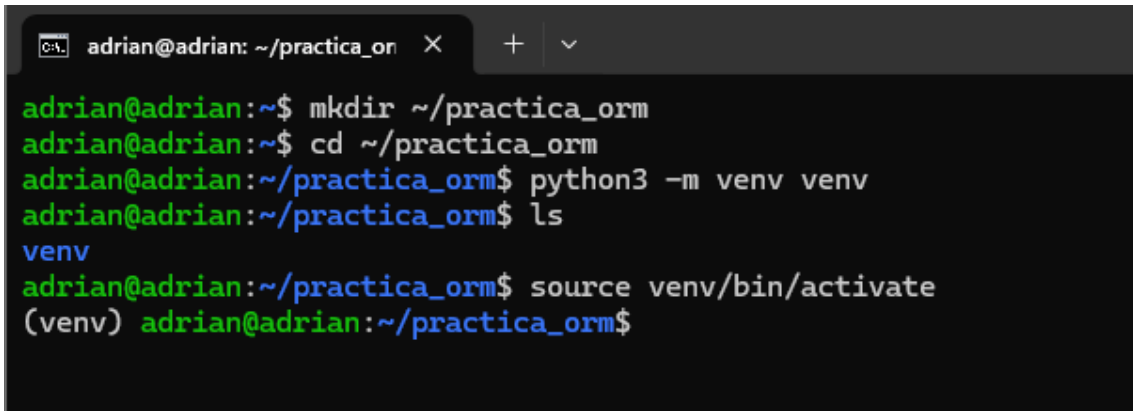


Figura 8. Creación y activación del entorno virtual de Python.

5.2 Dependencias de la aplicación

Instalé las dependencias necesarias y comprobé que SQLAlchemy y PyMySQL estaban disponibles dentro del entorno.

```
(venv) adrian@adrian:~/practica_orm$ pip install sqlalchemy pymysql
Collecting sqlalchemy
  Downloading sqlalchemy-2.1.5-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x
86_64.whl.metadata (9.7 kB)
Collecting pymysql
  Downloading pymysql-1.2.3-py3-none-any.whl.metadata (4.2 kB)
Collecting typing-extensions>=4.6.0 (from sqlalchemy)
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Downloading sqlalchemy-2.1.5-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86
_64.whl (4.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.6/4.6 MB 21.6 MB/s eta 0:00:00
Downloading pymysql-1.2.3-py3-none-any.whl (46 kB)
Downloading typing_extensions-4.16.0-py3-none-any.whl (45 kB)
Installing collected packages: typing-extensions, pymysql, sqlalchemy
Successfully installed pymysql-1.2.3 sqlalchemy-2.1.5 typing-extensions-4.16.0
(venv) adrian@adrian:~/practica_orm$
```

Figura 9. Instalación de dependencias Python.

```
(venv) adrian@adrian:~/practica_orm$ python -c "import sqlalchemy; print(sqlalchemy.__version__)"
2.1.5
(venv) adrian@adrian:~/practica_orm$ pip list
Package           Version
----------------- -------
pip               25.1.1
PyMySQL           1.2.3
SQLAlchemy        2.1.5
typing_extensions 4.16.0
(venv) adrian@adrian:~/practica_orm$
```

Figura 10. Comprobación de paquetes instalados en el entorno virtual.

6. Modelo ORM y creación de tablas

Definí una clase Alumno en SQLAlchemy, con atributos que se corresponden con columnas de la tabla relacional. A partir del modelo, SQLAlchemy generó la estructura necesaria en MariaDB.

6.1 Definición del modelo

El modelo de Python describe la entidad Alumno y sus atributos, que posteriormente se traducen a columnas de la tabla.

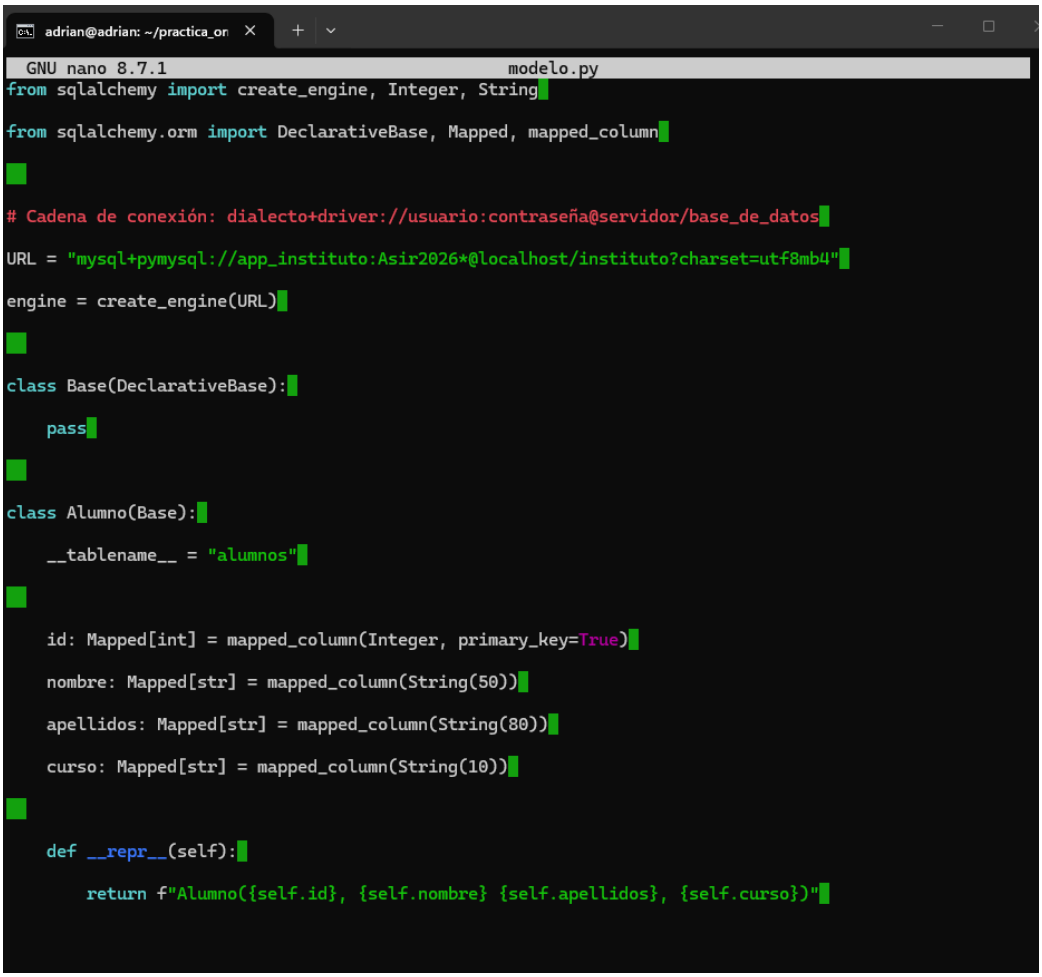


Figura 11. Definición del modelo Alumno con SQLAlchemy.

6.2 Creación y comprobación de la tabla

Ejecuté el script de creación y comprobé desde MariaDB la tabla alumnos generada por el ORM.

```
(venv) adrian@adrian:~/practica_orm$ python -c "import modelo; print('Modelo correcto')"
Modelo correcto
(venv) adrian@adrian:~/practica_orm$
```

Figura 12. Ejecución del script de creación del modelo.

```
(venv) adrian@adrian:~/practica_orm$ mariadb -u app_instituto -p instituto
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 38
Server version: 11.8.6-MariaDB-5ubuntu0.1 from Ubuntu -- Please help get to 10k stars at https://githu
b.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [instituto]> SHOW TABLES;
+---------------------+
| Tables_in_instituto |
+---------------------+
| alumnos             |
+---------------------+
1 row in set (0,002 sec)

MariaDB [instituto]>
MariaDB [instituto]> DESCRIBE alumnos;
+-----------+-------------+------+-----+---------+----------------+
| Field     | Type        | Null | Key | Default | Extra          |
+-----------+-------------+------+-----+---------+----------------+
| id        | int(11)     | NO   | PRI | NULL    | auto_increment |
| nombre    | varchar(50) | NO   |     | NULL    |                |
| apellidos | varchar(80) | NO   |     | NULL    |                |
| curso     | varchar(10) | NO   |     | NULL    |                |
+-----------+-------------+------+-----+---------+----------------+
4 rows in set (0,002 sec)

MariaDB [instituto]> exit
Bye
(venv) adrian@adrian:~/practica_orm$
```

Figura 13. Tabla alumnos creada por el ORM y estructura consultada desde MariaDB.

7. Persistencia y consulta de objetos

Creé objetos Alumno desde Python y los guardé mediante la sesión de SQLAlchemy, sin escribir manualmente sentencias INSERT. Después recuperé los registros utilizando el mismo modelo ORM.

7.1 Inserción de objetos

Preparé y ejecuté el script que crea instancias de Alumno y las persiste en la base de datos.

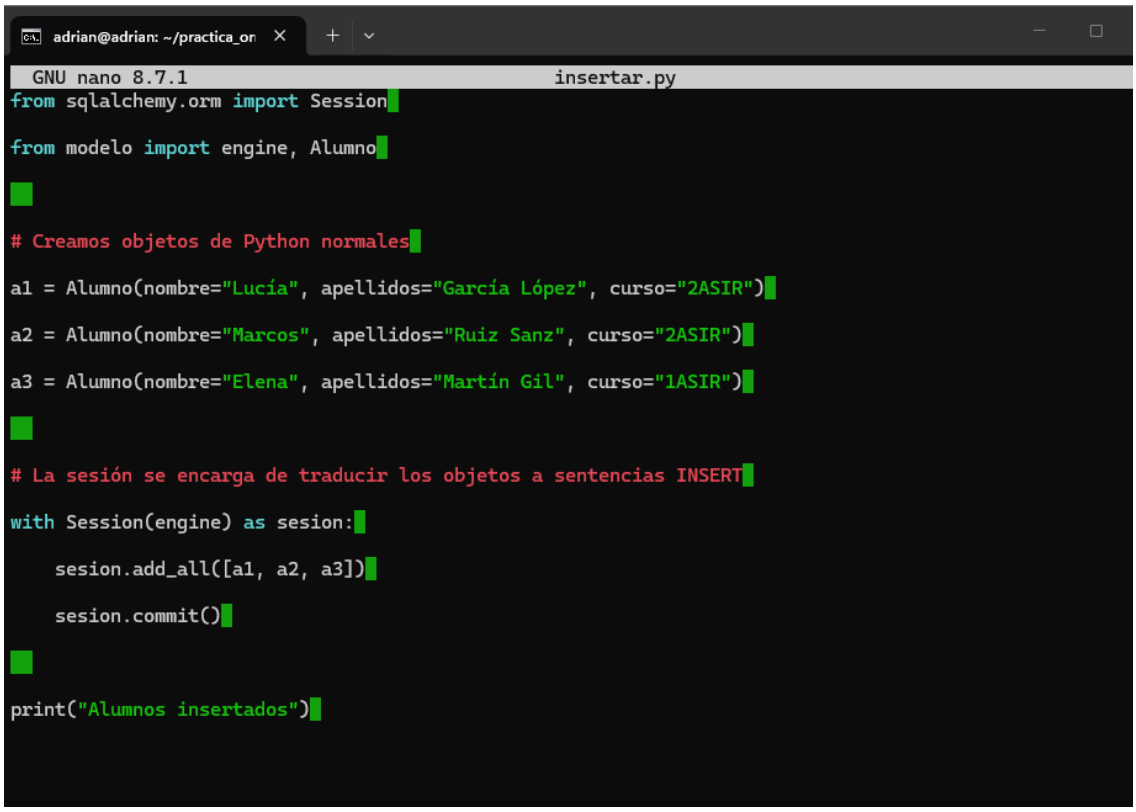


Figura 14. Script de inserción de objetos mediante SQLAlchemy.

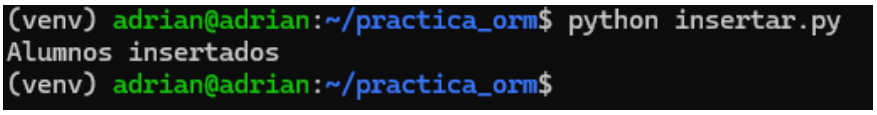


Figura 15. Ejecución del script de inserción.

7.2 Consulta mediante el ORM

Recuperé los objetos almacenados y comprobé sus valores desde Python.

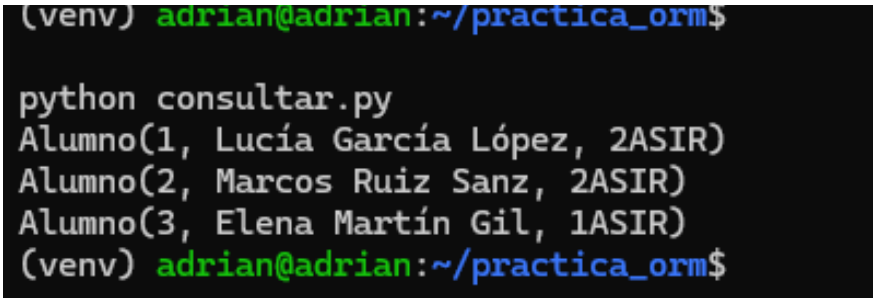


Figura 16. Objetos recuperados desde MariaDB mediante el ORM.

8. SQL generado y principio de mínimo privilegio

El registro general permite comprobar la traducción entre operaciones con objetos y SQL enviado al servidor. Además, la práctica valida que el usuario de aplicación dispone de los permisos necesarios, pero no de privilegios administrativos innecesarios.

8.1 Datos almacenados

Consulté directamente la base de datos para verificar los registros persistidos por el ORM.

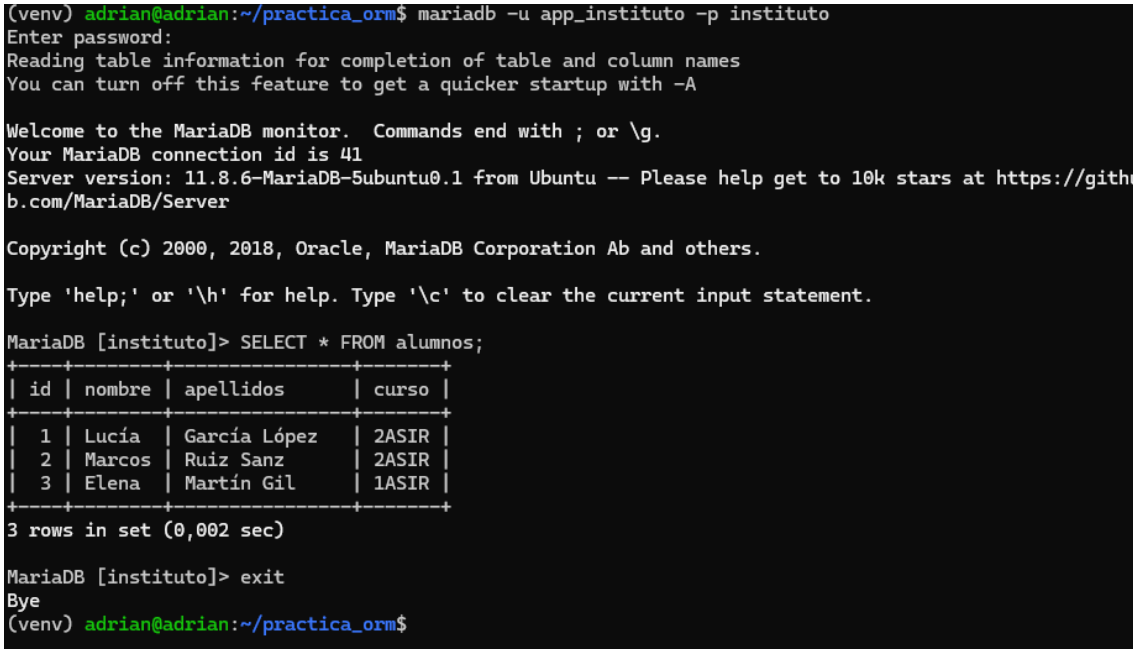


Figura 17. Datos almacenados vistos directamente desde MariaDB.

8.2 Validación de privilegios

Comprobé que el usuario de aplicación no podía ejecutar una operación destructiva no autorizada y que la tabla continuaba disponible.

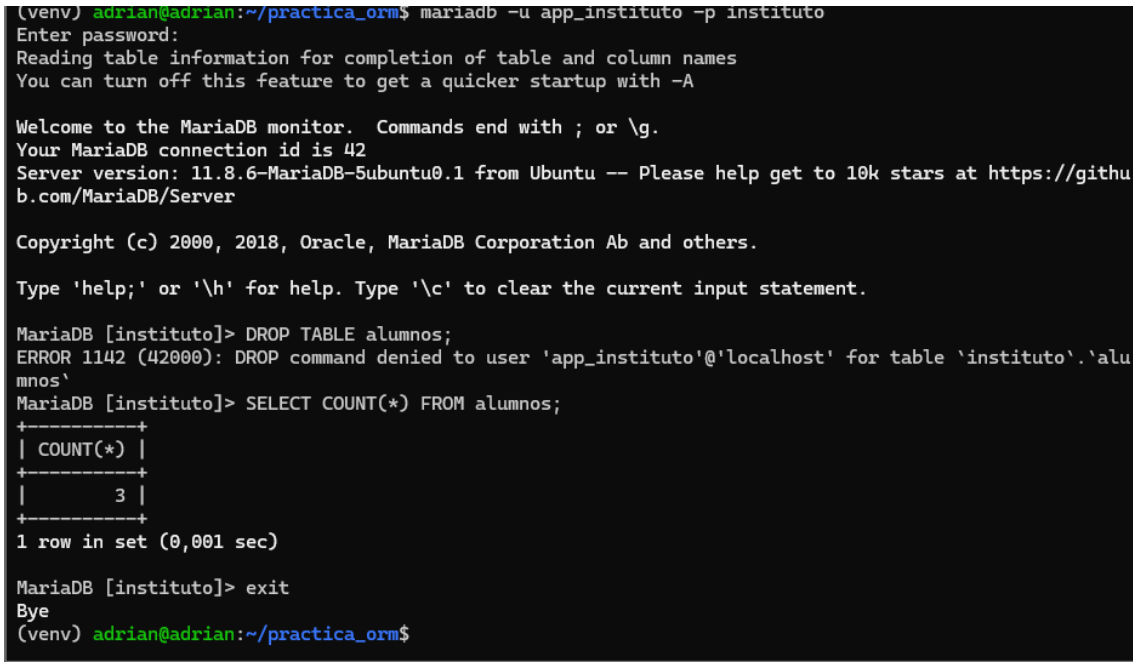


Figura 18. Operación DROP TABLE rechazada para el usuario de aplicación y tabla conservada.

8.3 Revisión del registro

Finalmente revisé el registro general para observar las operaciones enviadas por la aplicación antes de finalizar la práctica.

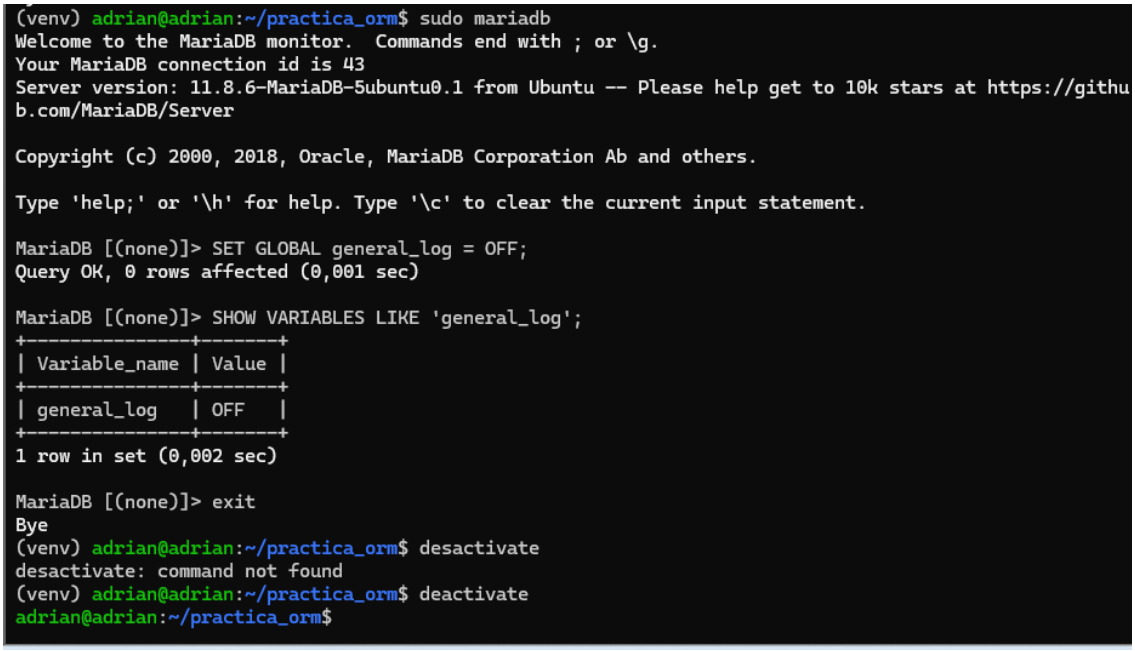


Figura 19. Comprobación final del registro general antes de su desactivación.

9. Resultado y competencias

- Preparación de MariaDB para una aplicación.
- Aplicación de privilegios limitados a un usuario técnico.
- Creación de entornos virtuales Python.
- Uso de SQLAlchemy y PyMySQL.
- Mapeo entre clase, atributos, objetos y tabla relacional.
- Creación, inserción y consulta mediante ORM.
- Observación del SQL generado en el servidor.
- Validación práctica del principio de mínimo privilegio.

10. Conclusión

La práctica permitió observar el ORM desde dos perspectivas complementarias: el código Python que trabaja con objetos y el servidor MariaDB que recibe y ejecuta las sentencias SQL resultantes. Esto permite relacionar desarrollo de aplicaciones con administración de bases de datos y control de permisos.